

FPGA Music Synthesizer: A Hardware-Realized Synthesizer with Embedded Emotion Classification

Luke Qiao, Aarav Wattal, and Joe Li
Stanford University, Department of Electrical Engineering

Abstract—This work presents a music synthesizer fully realized on a Field Programmable Gate Array (FPGA). Unlike traditional software-driven approaches, the system leverages application-specific hardware to efficiently deliver real-time audio synthesis. The synthesizer plays preloaded song sequences stored in ROM, supports up to three simultaneous notes (chords), and shapes timbre through additive harmonic synthesis and per-note Attack–Decay–Sustain–Release (ADSR) envelopes. A lightweight feedforward neural network (FNN) is integrated directly into the hardware pipeline to predict the emotion of the currently playing song, with inference performed on-FPGA and training performed off-FPGA. Real-time audio-visual feedback is provided through a VGA driver feeding an HDMI/DVI encoder, which renders the synthesized waveform on an external display and colors it according to the predicted emotion. By uniting digital signal processing (DSP), machine learning (ML), and interactive visualization within a single FPGA system, this project demonstrates the capability of FPGA platforms to serve as both real-time audio synthesizers and hardware accelerators for embedded ML tasks.

Index Terms—FPGA, Digital Signal Processing, Additive Harmonic Synthesis, Feedforward Neural Network, Hardware Acceleration, Real-Time Audio, VGA, ADSR.

I. PROJECT OVERVIEW

This project presents the design and implementation of a music synthesizer fully realized on a Field Programmable Gate Array (FPGA). Unlike traditional software-driven approaches, this system leverages application-specific hardware to efficiently deliver audio synthesis. A demo of the system is available [here](#).

The main objectives of this project are:

- Develop a high-performance music synthesizer with preloaded song sequences implemented purely in hardware.
- Integrate machine learning (ML) for emotion prediction into the hardware pipeline.
- Demonstrate advanced digital signal processing (DSP) and sound synthesis techniques with hardware efficiency.
- Provide real-time audio-visual feedback through an external display controlled using a VGA driver feeding into an HDMI/DVI encoder.

The main features of the system are:

- **Preloaded Songs:** The synthesizer plays preprogrammed songs stored in ROM, controlled by the play and pause buttons.
- **Emotion Prediction:** A feedforward neural network (FNN) implemented in hardware predicts the emotion of the current song. The FNN architecture is lightweight,

making it suitable for processing one-dimensional data streams while ensuring efficient inference within FPGA resource constraints. The predicted emotion is displayed as the color of the waveform on the external display.

- **Chords:** The ability to play multiple notes simultaneously, enabling the synthesizer to output harmonies. The duration of each note in a chord can be individually controlled with a custom ROM protocol to achieve the illusion of multiple musical lines.
- **DSP & Harmonic Synthesis:** Sound generation leverages hardware DSP blocks for precise waveform shaping and additive harmonic synthesis, supporting multiple harmonics to generate characteristic timbres. Each note follows an Attack, Decay, Sustain, and Release (ADSR) envelope implemented as hardware-controlled modules, providing fine-grained control of note dynamics that simulates acoustic instrument behavior.
- **Read-Only Memory (ROM):** Preloaded waveform samples for sine waves and lookup tables are stored in on-chip ROMs, facilitating efficient retrieval without significant memory overhead and ensuring rapid access for high-frequency audio synthesis.
- **Random-Access Memory (RAM):** A dual-port RAM supports the waveform-display module, allowing modules to write to and read from memory independently. To prevent read/write hazards at the same address, double buffering splits the RAM addresses in half and tracks a read-index bit that determines which half is read from or written to, ensuring no two modules access the same half in a single clock cycle.
- **VGA Driver for External Screen:** An integrated video controller outputs the synthesized waveform and emotion states via a VGA driver feeding into an HDMI/DVI encoder, providing an intuitive interface for both performance and debugging.

This project demonstrates the capability of FPGA platforms to serve not only as real-time audio synthesizers but also as hardware accelerators for embedded machine learning tasks. By uniting DSP, ML, and interactive visualization within a single FPGA system, it highlights the potential for scalable, low-latency, and adaptive digital instruments.

II. HARDWARE OVERVIEW

The project uses the built-in Xilinx FPGA buttons and audio jack along with an external screen. On-board LEDs light up when the synthesizer is playing notes. The board provides

dedicated buttons for reset, play, and advancing to the next song. The board layout is shown in Fig. 1.

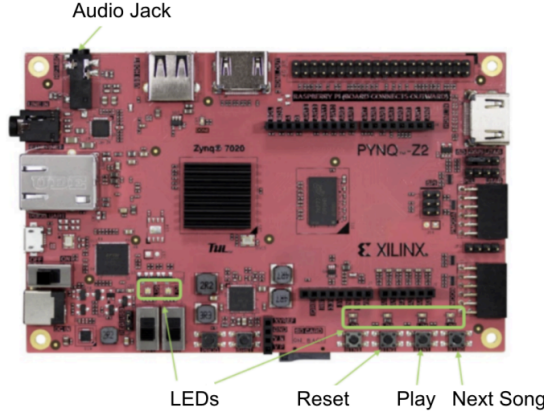


Fig. 1. FPGA board layout showing the audio jack, LEDs, and the reset, play, and next-song buttons.

III. HIGH-LEVEL DESIGN

The `top_level` module contains the `music_player` module, which is responsible for handling the logic for the current song and determining which notes to play. The output of the `music_player` module is fed into the `wave_display_top` module to drive the screen, while the audio output is fed into the `adau1761_codec` conditioner to drive the speakers. The overall top-level architecture is shown in Fig. 2.

The pipelining of the `music_player` module is: `song_reader` $\rightarrow$ `note_player_selector` $\rightarrow$ `adau1761_codec`, as shown in Fig. 3.

The pipelining of the `note_player_selector` module is `note_player` (3x) $\rightarrow$ `frequency_rom` $\rightarrow$ `sine_reader` (4x) $\rightarrow$ `sine_rom`, `dynamic_rom`, as shown in Fig. 4.

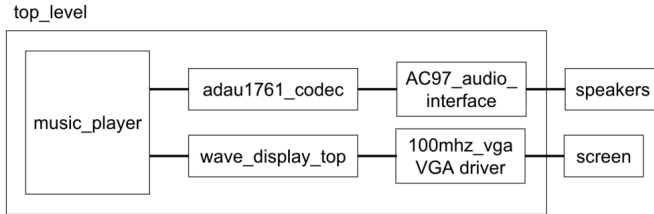


Fig. 2. Top-level block diagram.

IV. CORE SYSTEM MODULES

A. Music Player

The `music_player` module is the highest-level module relating to synthesizing the proper notes to be displayed on the screen and played on the speakers.

music_player

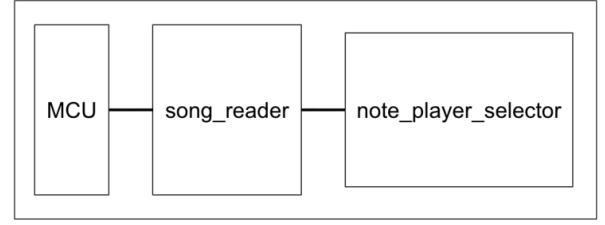


Fig. 3. Music player block diagram.

note_player_selector

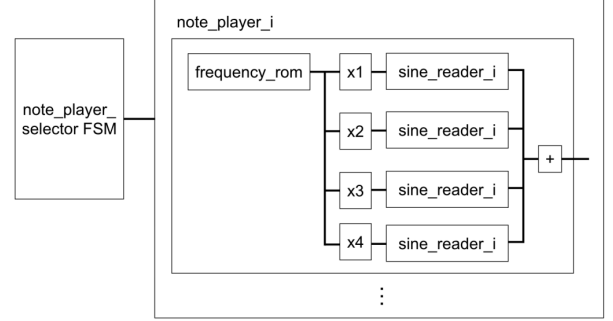


Fig. 4. Note player selector block diagram. Note that $i \in [1, 3]$.

1) *Music Player Testbench:* The test strategy was to let the music player module run and record the output waveform. We show the first 4 million nanoseconds of the simulation for the sake of space. After the play button is pressed, the output samples begin to generate the audio signal, as expected. The signal looks more complicated than a pure sine wave because three different note frequencies in the chord are added together, in addition to four harmonics per note. The testbench output waveform is shown in Fig. 5.

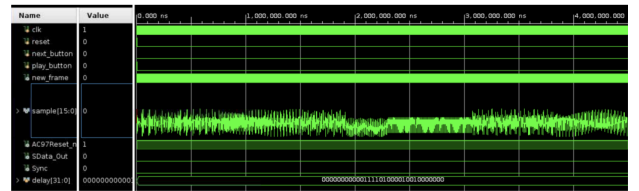


Fig. 5. Music player testbench output waveform.

B. MCU

The MCU is a sub-module of `music_player`. It initially starts in its reset state. When the play button is pressed, the MCU signals to begin playing the current song. When the song finishes, the MCU waits in the paused state at the beginning of the next song. If the next button is pressed, the MCU instructs the `song_reader` to go to the beginning of the next song and pause. The MCU finite state machine is shown in Fig. 6.

1) *MCU Testbench:* When the play button is pressed, the play output goes high until the `song_done` signal goes

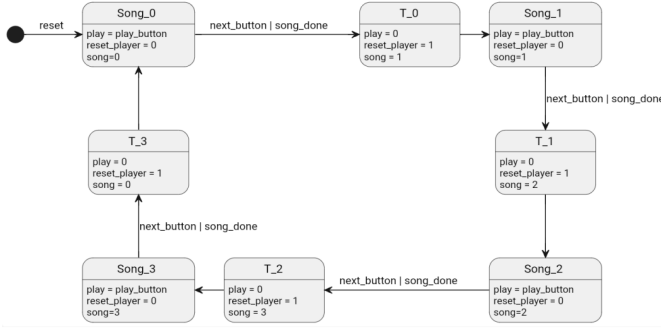


Fig. 6. MCU finite state machine.

high, at which point `play` goes low and `reset_player` goes high, as expected, since the MCU should reset the player and start paused at the next song. The song index is correctly incremented every time `song_done` goes high. This behavior is shown in Fig. 7.

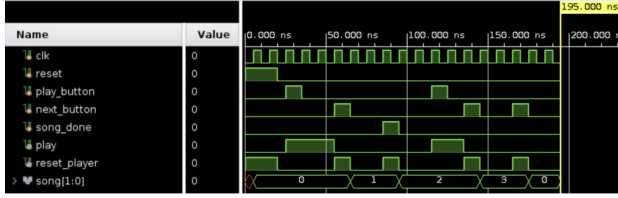


Fig. 7. MCU testbench waveform.

C. Song Reader

The `song_reader` is responsible for determining which note to play given the provided `song_rom`, generated using a custom script. The `song_reader` takes in the current song to read from the MCU and reads from the `song_rom` (this reading operation takes one clock cycle). It then sends the new note and duration to the `note_player`. The `song_reader` waits for the `note_player` to signal that it has finished the note. When the `note_player` finishes the current note, the `song_reader` looks up the next note and sends it to the `note_player`. This repeats until the `song_reader` finishes the current song, at which point it signals the MCU that the song is done. The song reader FSM is shown in Fig. 8.

1) *Song Reader Testbench*: The testing strategy was to let the song reader module run and record the output waveform. We show the first 2 million nanoseconds of the simulation. We verify that the note, duration, and metadata shown in the waveform match the entries of the song ROM. The long periods where note and duration are constant correspond to the “wait” entries in the song ROM, where the song reader waits for the note players to finish their current notes. As expected, the note changes every time `new_note` goes high for one clock cycle. The `currently_used` output of each note player correctly reflects when each note player is playing a note of the chord, and output samples are generated when `generate_next_sample` is high. This is shown in Fig. 9.

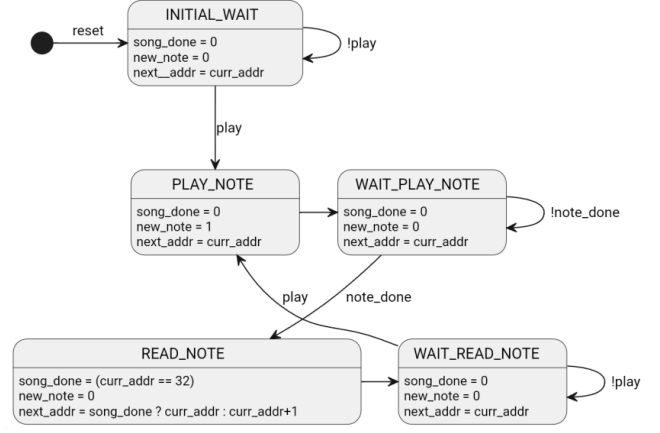


Fig. 8. Song reader FSM. Note that for all states: $\{note, duration\} = ROM[\{song, curr_addr\}]$.

A zoomed-in version of the output waveform, shown in Fig. 10, shows that the shorter constant-note periods correspond to individual notes read from the `song_rom` and fed into the three note players. From the `note_player_to_use` waveform, `note_player_1` takes the first note, `note_player_2` the second, and `note_player_3` the third, as expected, and the `currently_used` waveforms reflect this behavior.

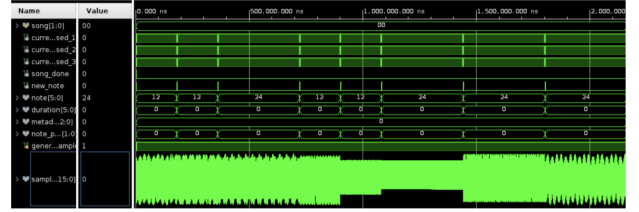


Fig. 9. Song reader testbench waveform.

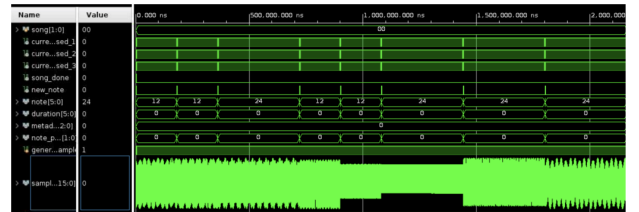


Fig. 10. Song reader testbench zoomed in waveform.

D. Note Player

The `note_player` is responsible for playing a single note with a given duration and pitch. This is done by looking up the step size required to generate the correct frequency for that note and then synthesizing a sine wave at that frequency for as long as the note is playing. The `note_player` also requires a counter to track the duration of the current note. This counter ticks every 48th of a second while the note is

playing; if the song is paused, the counter pauses as well. The 48th beats are generated by a `beat_generator` module.

1) *Note Player Testbench:* Our testing strategy is to let the note player module run for multiple values of metadata (encoding different frequency spectra) and verify that the output sample has the correct audio signal shape. In all waveforms, the signal lasts for 5 beats and becomes 0 afterwards, as expected.

- Metadata = 0 represents a pure sine wave, which we observe, as shown in Fig. 11.
- Metadata = 1 represents a square wave, which we observe is approximated, as shown in Fig. 12.
- Metadata = 2 represents a sawtooth wave, which we observe is approximated, as shown in Fig. 13.

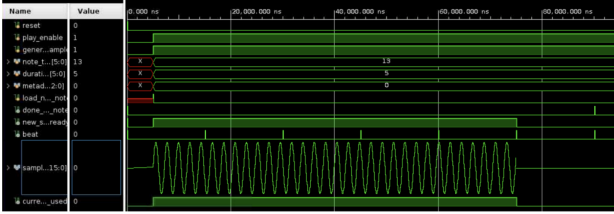


Fig. 11. Note player testbench waveform for sine wave metadata.

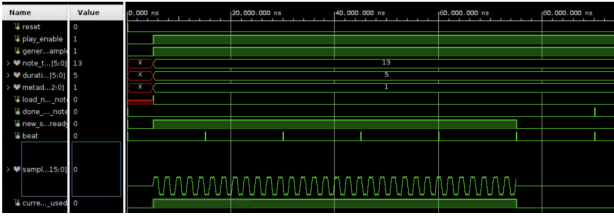


Fig. 12. Note player testbench waveform for square wave metadata.

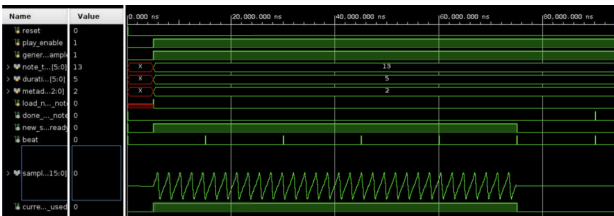


Fig. 13. Note player testbench waveform for sawtooth wave metadata.

E. Sine Reader

The `sine_reader` is a sub-module of the `note_player` that takes in a step size (determined by the frequency of the note) and repeatedly reads from the `sine_rom`, generating a sine wave whose frequency is determined by the step size. The `sine_rom` takes in an address from `sine_reader` and produces a sample of the sine wave. By addressing the sine ROM correctly, we can generate sine waves of arbitrary frequency. For each next sample, we increment the address pointer into the `sine_rom`

by a step size representing frequency. A larger step size traverses the sine wave faster, generating a higher frequency, and vice versa. The sine reader block diagram is shown in Fig. 14.

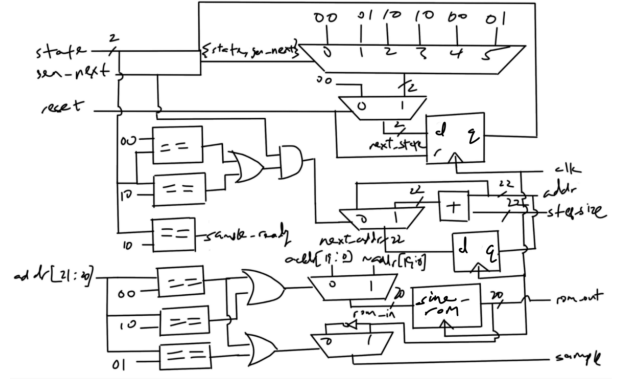


Fig. 14. Sine reader block diagram.

1) *Sine Reader Testbench:* We test the output of `sine_reader` and verify that it is a sine wave. A step size of $\{10' d168, 10' d983\}$ was used to control the frequency of the output sample. The output is shown in Fig. 15.

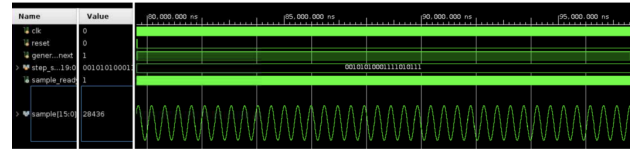


Fig. 15. Sine reader testbench waveform.

F. Wave Display Top

The top-level `wave_display_top` module is responsible for displaying the waveform on the external screen. It is composed of two sub-modules: (1) the `wave_capture` module, which collects the individual samples output by the `music_player` and reassembles them in a RAM, and (2) the `wave_display` module, which is given screen coordinates by the VGA driver and determines the color of the pixel at that coordinate by reading from the RAM. The top-level block diagram is shown in Fig. 16.

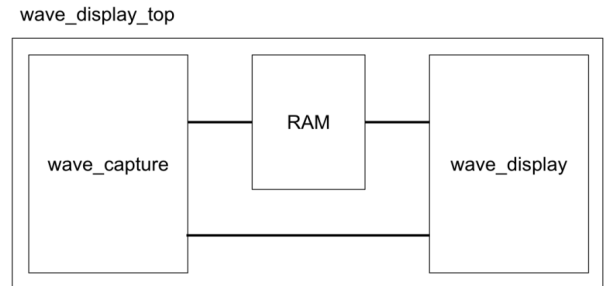


Fig. 16. Wave display top-level block diagram.

G. Wave Capture

The `wave_capture` module implements an FSM that handles the audio samples from the `music_player` module. The FSM is initially in the armed state. After a zero crossing, the FSM switches to the active state, where it stores the 8 most significant bits of the next 256 audio samples into the RAM. Once it writes the final sample, it moves into the wait state, which ensures that the `read_index` bit for double buffering is not flipped prematurely. The wave capture FSM is shown in Fig. 17.

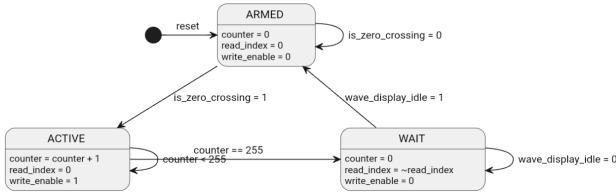


Fig. 17. Wave capture FSM. Note that for all states: $\text{write_sample} = \text{sample}[15:8] \oplus 8'd128$, $\text{write_addr} = \{\text{read_index}, \text{counter}\}$.

1) *Wave Capture Testbench.* One clock cycle after `dout` shifts sign, write enable turns on. Here, `dout` shifts from positive to negative, but the module reverses the first bit of `dout`, so it effectively goes from negative to positive. Write enable turns on as soon as this sign change happens, and `write_sample` is correct, starting to count from zero as soon as the sign change happens. This is shown in Fig. 18.

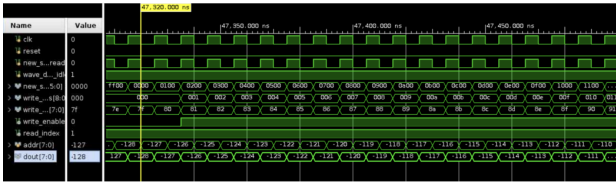


Fig. 18. Wave capture testbench waveform.

H. Wave Display

The `wave_display` module is responsible for displaying the waveform onto the raster display. On every clock cycle, `wave_display` is given the X (11 bits) and Y (10 bits) coordinates of a screen pixel from the VGA driver, and on the next cycle it outputs the color of that pixel. At any coordinate (X, Y) , the pixel should be colored if the Y value lies between $\text{RAM}[X - 1]$ and $\text{RAM}[X]$. The VGA sweeping pattern over screen coordinates is shown in Fig. 19.

We must convert the given X and Y coordinates to the proper 9-bit RAM addresses to correctly render the waveform. To make the waveform more visible, we drop the least significant bits of X and Y . Then, to draw the waveform in the correct quadrants of the display, we follow the mappings below.

We do not care when the X coordinate is in the 1st or 4th quadrants, and we do not care when the Y coordinate is on the

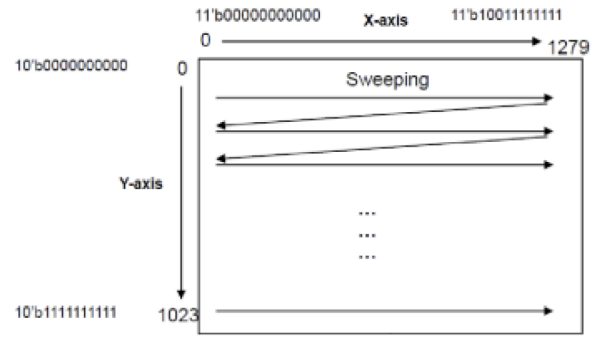


Fig. 19. Wave display diagram.

TABLE I
X-COORDINATE TO READ-ADDRESS MAPPING.

$X[10:0]$	read_addr
11'b000xxxxxxx	DNC
11'b001xxxxxxx	{read_index, 8'b0xxxxxxx}
11'b010xxxxxxx	{read_index, 8'b1xxxxxxx}
11'b011xxxxxxx	DNC

TABLE II
Y-COORDINATE TO READ-ADDRESS MAPPING.

Y[10 : 0]	read_addr
10'b0xxxxxxxxx	{read_index, 8'bxxxxxxxxx}
10'b1xxxxxxxxx	DNC

bottom half of the screen, denoted by “DNC” in the mapping. The valid pixel locations and the corresponding wave display block diagram are shown in Fig. 20 and Fig. 21, respectively.

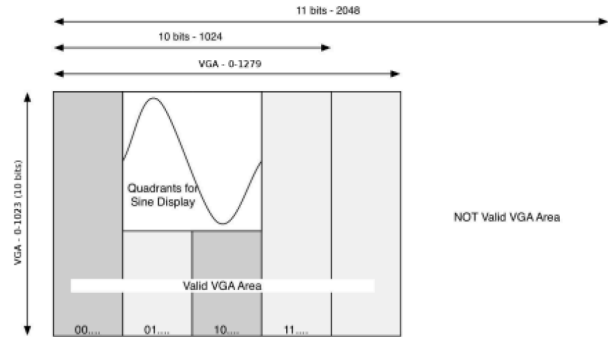


Fig. 20. Wave display valid pixel locations.

1) *Wave Display Testbench:* In the testbench, `x_sim` and `y_sim` continually change due to nested for-loops, with a new combination occurring once per clock cycle. The RAM address changes every two x increments, visible on the waveform diagram. In this specific example, the RGB values do not change because the simulation window is not long enough, but they change over time when the simulation is extended. Additionally, `valid_pixel_sim` is high only when `valid` is high and the coordinate is in the desired quadrants (X and Y both in valid ranges). This is shown in Fig. 22.

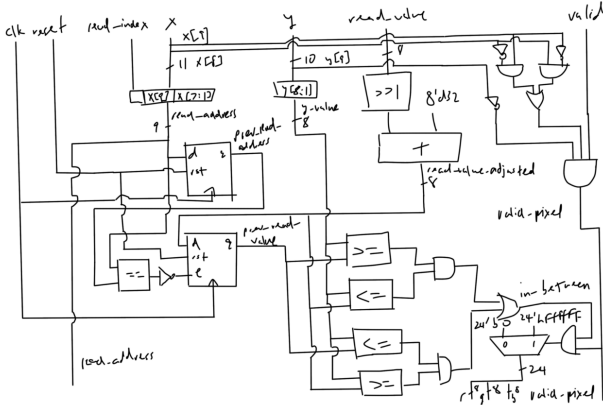


Fig. 21. Wave display block diagram.

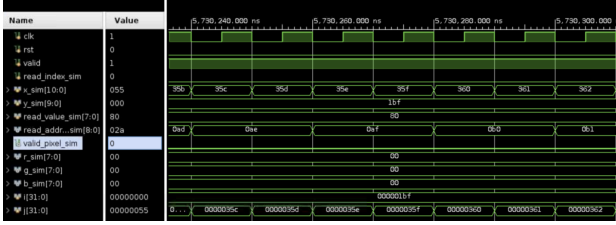


Fig. 22. Wave display testbench waveform.

I. Chords

Chords involve playing multiple notes at the same time. The design can play up to 3 notes simultaneously by instantiating 3 note_player modules in note_player_selector. We use the following format for entries in the song_rom to support chords. The chords block diagram is shown in Fig. 23.

TABLE III
SONG ROM FORMAT FOR CHORD SUPPORT.

ROM format	Meaning
0 xxxxxx yyyyyy zzz	Schedule note xxxxxx (6 bits) to immediately start playing for a duration of yyyyyy (6 bits) 48th notes. The 3 zzz bits are metadata encoding timbre and harmonics. We use 48th notes as the time basis to allow both 16th notes and triplets to be encoded. After scheduling, move to the next item in the song_rom.
1 yyyyyy	Wait yyyyyy (6 bits) 48th notes before reading the next entry in the song_rom.

1) *Note Player Selector FSM*: The note player selector FSM coordinates assignment of incoming notes to the three available note players and handles rests and address incrementing. It is shown in Fig. 24.

2) *Note Player Selector Testbench*: We test this module by manually feeding in 3 different notes with different durations and specifying which note player to use via the note_player_to_use input. The first note player plays for 2 beats, the second for 4 beats, and the third for 2 beats, as expected, verifiable in the currently_used outputs of each note player. The output sample is the superposition of

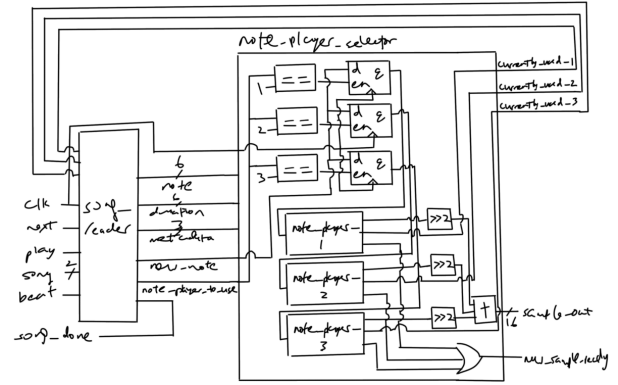


Fig. 23. Chords block diagram.

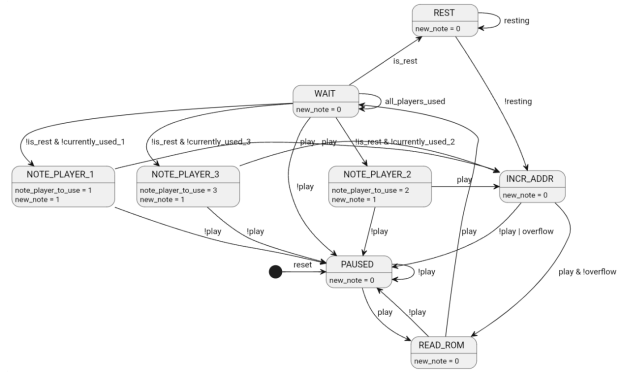


Fig. 24. Note player selector FSM.

all three note-player audio signals. In regions where multiple note players play at once, the output signal appears more complicated; in regions where exactly one note player is active, the output signal is a pure sine wave at the correct frequency. This is shown in Fig. 25.

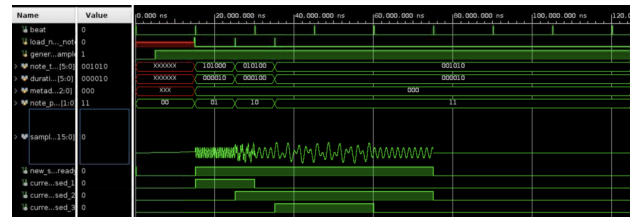


Fig. 25. Note player selector testbench waveform.

J. Music Emotion Classifier

The music_emotion_classifier module implements a feedforward neural network (FNN) that uses the last 2 chords played to predict the sentiment of the song (happy, sad, angry) at that point in time. We integrate an FNN from scratch, where training is done off-FPGA and inference is computed on-FPGA. We first receive an input vector of the notes played, which is loaded into the processing pipeline and fed into the first layer of the network. Each layer contains

multiple neuron modules; each neuron multiplies the input by weights, adds a bias, and applies an activation function. Weight and bias values are preloaded into memory blocks so each neuron has the proper parameters for computation. Finally, the output of the last layer is converted into a classification result. We represent happy with green, sad with blue, and angry with red, and visualize the output as a weighted sum of these colors on the output wave. The neuron block diagram, the high-level FNN block diagram, and the FNN finite state machine are shown in Fig. 26, Fig. 27, and Fig. 28, respectively.

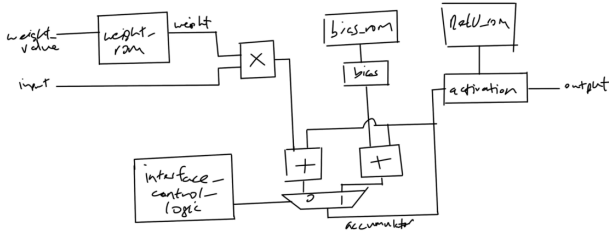


Fig. 26. Neuron block diagram.

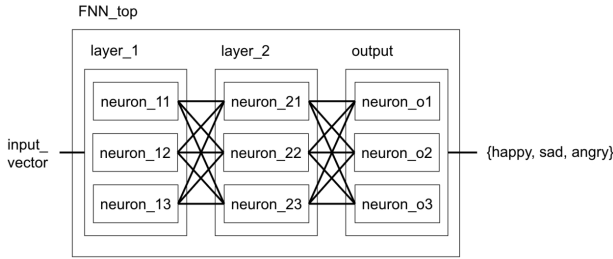


Fig. 27. FNN high-level block diagram.

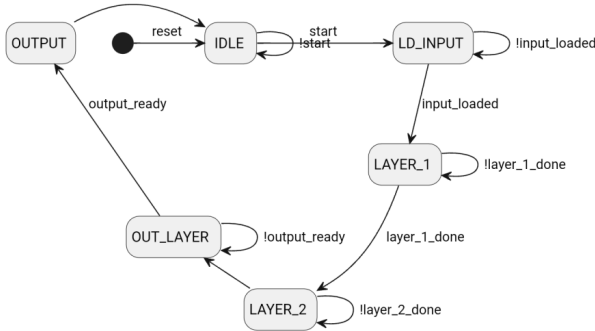


Fig. 28. FNN finite state machine.

1) *Neuron Testbench:* In this testbench, the neuron multiplies activation index 0 by weight 0 to get 2, activation index 1 by weight 1 to get 6, and activation index 2 by weight 2 to get -3, then sums these products to get 5, as expected. Adding the bias of 1 gives a total weighted sum of 6, so the ReLU activation outputs 6 because it is non-negative. If the value fed into the ReLU were negative, the neuron would output 0. This is shown in Fig. 29.

```
# run 1000ns
t=0 state=00 i=0 act=0 w=1 bias=1 acc=0 done=0 out=0
t=10000 state=00 i=0 act=2 w=1 bias=1 acc=0 done=0 out=0
t=25000 state=01 i=0 act=2 w=1 bias=1 acc=0 done=0 out=0
t=35000 state=01 i=1 act=3 w=2 bias=1 acc=2 done=0 out=2
t=45000 state=01 i=2 act=3 w=-1 bias=1 acc=8 done=0 out=8
t=55000 state=01 i=3 act=0 w=3 bias=1 acc=5 done=0 out=5
t=65000 state=10 i=3 act=0 w=3 bias=1 acc=6 done=0 out=6
t=75000 state=00 i=3 act=0 w=3 bias=1 acc=6 done=1 out=6
Final Neuron output: 6
```

Fig. 29. Neuron testbench output.

2) *Music Emotion Classifier Testbench:* The full music_emotion_classifier is tested after being integrated into the music synthesizer system. The expected output matches the output-layer outputs, as the hex values of the expected output convert exactly to the output-layer outputs. All computations within the hidden layers are also computed properly, as shown in Fig. 30.

```
Classification complete for C4 to E4:
Output: 3600ef
Output values: [ 54, 0, 239]
Hidden Layer 1 outputs: [ 99, 16, 0]
Hidden Layer 2 outputs: [ 118, -107, -12]
Output Layer outputs: [ 54, 0, 239]
```

```
Test Case 2: G4 to B4
Classification complete for G4 to B4:
Output: 8b00b4
Output values: [139, 0, 180]
Hidden Layer 1 outputs: [ 0, 11, 0]
Hidden Layer 2 outputs: [ 44, 4, -42]
Output Layer outputs: [139, 0, 180]
```

Fig. 30. Music emotion classifier testbench output.

K. Harmonics

Harmonics involve playing multiple overtones over the fundamental frequency of each note. In the design, we play the first 4 terms in the harmonic series (the first 4 integer multiples of the fundamental frequency) and tune the weights using Fourier analysis to approximate square and sawtooth waves for different timbres. These weights are stored in harmonic_rom. The output wave is a weighted sum of each term in the harmonic series. The harmonics block diagram is shown in Fig. 31.

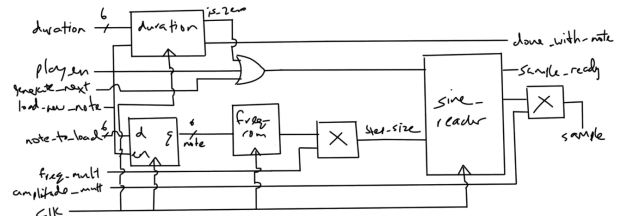


Fig. 31. Harmonics block diagram.

L. Dynamics

Dynamics involve creating an ADSR envelope for each output note, implemented by modifying the `sine_reader` module. We opted for a very fast rise time (essentially instant), a gradual exponential decay, and a short sustain period. The exponential decay was implemented using an `exponential_rom` module that contains 4096 sampled values from a decaying exponential function with tuned parameters. The exponential envelope has a delay of 1.5 s and a decay rate of 1 s^{-1} . Note that, for clarity, the testbench waveforms throughout this document are zoomed, making the time scale too short to visibly show a large effect from the ADSR envelope.

V. KEY IMPLEMENTATION DETAILS

Several implementation details proved important. We used flip-flops to reduce the length of the critical path and resolve timing errors; without these flip-flops, the critical path (from the `sine_reader` output to the output of the `song_reader` module) would be too long for the 100 MHz clock. We also kept the neural network separate from the music player, implementing both independently and integrating them at the end. Minimizing the interaction between the neural-network code and the music-player code made the codebase easier to maintain and debug, and ensured that introducing the neural network did not create unexpected behavior. We implemented dynamics at the `sine_reader` level, the lowest level in the design hierarchy, so the ADSR envelope applies to every note played. Finally, we drew inspiration from our chord implementation to implement harmonics.

VI. PROBLEMS ENCOUNTERED

The main problems encountered were timing errors. As mentioned, we resolved the setup-time error after synthesis by adding flip-flops to break up the critical path. We also encountered errors where values changed at the wrong time (i.e., too early), which we diagnosed by carefully examining the waveforms from testbench simulation. Finally, at the system-design level, we ran into numerous integration errors, as the entire system would not work when run at the top level. We debugged these by starting at lower levels of the design (often `note_player` or `sine_reader`), confirming correctness via testbench outputs, and moving up the hierarchy toward higher-level modules.

REFERENCES

- [1] J. O. Smith, *Spectral Audio Signal Processing*. W3K Publishing, 2011.
- [2] N. H. Fletcher and T. D. Rossing, *The Physics of Musical Instruments*. Springer, 1998.
- [3] Xilinx Inc., “PYNQ-Z2 Reference Manual,” 2018.